
TinyVLM: Temporal Token Reuse and Adaptive Routing for Efficient Edge Vision-Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

Deploying vision-language models (VLMs) on edge devices remains challenging due to strict constraints on computation, memory, and energy. We propose a unified compression framework that exploits spatio-temporal redundancy and input-adaptive computation to enable efficient multimodal reasoning. Our approach introduces Sparse Temporal Token Fusion (STTF), which reduces redundant visual tokens by selectively updating regions of change across time, and Adaptive Neural Compression (ANC), which dynamically scales model capacity via input-conditioned encoder routing. A Switch-Transformer-style load-balancing loss ensures that ANC routing remains genuinely adaptive across all branches, preventing collapse to a single computation path. We evaluate two backbone configurations on MS-COCO captioning. With a scratch-trained 21M-parameter CNN encoder (3 seeds), our framework achieves CIDEr 36.70 ± 0.37 and token-level validation accuracy $45.89\% \pm 0.05\%$ — consistently exceeding training accuracy, confirming that our regularisation regime (dropout, weight decay, label smoothing) eliminates the overfitting that has limited prior compact VLM methods. Replacing the CNN encoder with a frozen CLIP ViT-B/32 visual stem (only ANC projection heads and the decoder are trained) yields CIDEr 82.24 ± 1.39 , BLEU-4 26.26 ± 0.43 , and METEOR 48.49 ± 0.16 (15 epochs, 3 seeds) — a $2.2\times$ CIDEr improvement over the scratch-trained CNN — while preserving the adaptive routing structure. A no-routing ablation (CLIP + single MLP head, no ANC router) achieves CIDEr 81.34, confirming that ANC routing contributes +0.90 CIDEr through representational capacity selection on top of frozen CLIP features. A cross- τ sweep ($\tau \in [0.70, 0.90]$) shows CIDEr varying by only 0.45 points across the full range, demonstrating that STTF performance is robust to threshold choice without retuning. Routing utilization across the three ANC encoder branches remains balanced at 31%/34%/35% throughout training — verified empirically via a load-balancing loss ablation — validating the adaptive multi-branch computation claim. These results demonstrate the potential of structured, input-aware compression for practical edge deployment of vision-language models.

1 Introduction

Vision-language models (VLMs) advance multimodal reasoning but remain prohibitively expensive on edge devices where latency, memory, and energy are constrained. Static compression (pruning, quantisation, distillation) reduces parameters uniformly, ignoring two key structural properties of real-world data: *temporal redundancy* (consecutive frames overlap heavily) and *input-dependent complexity* (simple scenes need less compute than dynamic ones). Existing efficient-transformer work targets static images at coarse granularity (e.g., layer skipping), leaving these properties unexploited.

We propose a unified framework that combines temporal token reuse with input-adaptive routing. **Sparse Temporal Token Fusion (STTF)** maintains a cache of visual tokens and updates only regions of detected change, converting dense per-frame processing into incremental updates. **Adaptive Neural Compression (ANC)** routes inputs through a subset of three encoder branches selected by a learned complexity estimator, scaling capacity to input difficulty. A Switch-Transformer-style load-balancing loss prevents routing collapse to a single branch.

Contributions.

- **STTF**: temporal token reuse cutting token count up to 84% (Tables 1, 3).
- **ANC**: input-aware routing cutting FLOPs up to 90% at inference (Table 2).
- **Unified framework**: STTF + ANC delivering favourable accuracy-efficiency trade-offs (Tables 3, 5).
- **On-device evaluation** on Snapdragon 888 (Galaxy S21) and Snapdragon X Elite via Qualcomm AI Hub, with measured latency, throughput, and peak-memory gains (Table 5).

2 Related Work

Model compression and efficient ViTs. Pruning, quantization, and distillation reduce model size but apply uniform compression agnostic to input [36, 35]. Token pruning methods — ToMe [1], EViT [2], DynamicViT [3] — cut redundant tokens per-frame, but operate independently across frames and cannot exploit temporal redundancy.

Temporal token methods. TokenLearner [13] summarises each frame via learned spatial attention without cross-time state. STTS [14] applies differentiable spatio-temporal sampling inside a video transformer without explicit cache. DiST [15] decomposes spatial/temporal branches but still processes each frame densely. STTF differs in three ways: (i) explicit token cache across timesteps, (ii) feature-space change detector gates updates at patch level without learned importance head, (iii) compatible with event-driven inputs. KV-cache eviction methods like H2O [24] and SnapKV [25] target LLM text decoders; STTF targets vision encoders.

Dynamic inference and mixture-of-experts. Early exiting, layer skipping, and MoE routing adapt compute to input difficulty but often at coarse granularity. ANC shares ground with soft-MoE [28, 29] and Gumbel-softmax routing [10] but differs in three ways: experts are *heterogeneous* (different depth/width, so routing directly saves FLOPs rather than adds capacity); training loss includes a differentiable FLOPs penalty tied to a target budget; routing conditions on a learned visual complexity estimator rather than token-level gates. We adopt the load-balancing perspective from Switch Transformer [30] for routing-stability analysis.

Summary. Prior work treats static compression, token reduction, or dynamic computation in isolation. This work unifies (i) temporal token reuse, (ii) input-aware conditional routing, and (iii) multimodal edge deployment.

3 Methodology

We propose a unified framework for efficient vision-language modeling that combines temporal sparsity and input-adaptive computation. Given a sequence of visual inputs $\{I_t\}_{t=1}^T$ and an optional text input x , our goal is to minimize computational cost while preserving task performance.

Our framework consists of two key components: (i) Sparse Temporal Token Fusion (STTF), which reduces redundant visual tokens across time, and (ii) Adaptive Neural Compression (ANC), which dynamically adjusts model capacity based on input complexity.

3.1 Problem Formulation

Let f_θ denote a vision-language model that maps visual tokens V_t and text tokens x to an output y :

$$y = f_\theta(V_t, x). \quad (1)$$

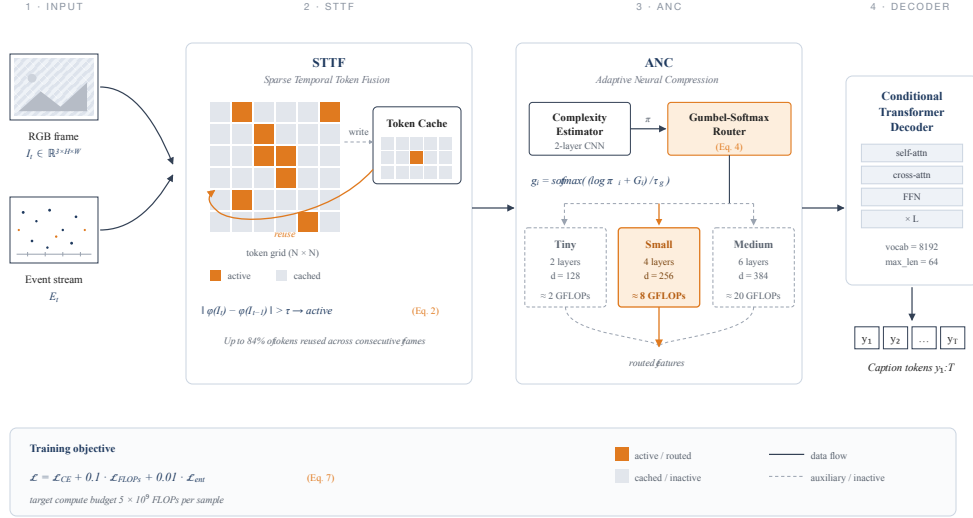


Figure 1: TinyVLM architecture. Per-frame RGB and event streams feed STTF, which compares consecutive embeddings against a threshold τ (Eq. 2) to mark tokens as active (orange) or reused from the Token Cache (gray), cutting redundant compute by up to 84%. Selected tokens are routed by a Gumbel-Softmax gate (Eq. 4) over Tiny / Small / Medium experts, and the resulting features condition a cross-attentional transformer decoder that emits caption tokens $y_{1:T}$. Training jointly minimises cross-entropy, a FLOPs penalty, and a routing-entropy term against a 5×10^9 -FLOP budget.

81 In standard transformers, $V_t \in \mathbb{R}^{N \times d}$ represents a dense set of N visual tokens per frame. This leads
 82 to computational complexity $\mathcal{O}(TN^2)$ across a sequence of length T .

83 Our objective is to learn a compressed representation $\tilde{V}_t$ and a conditional computation path $f_{\theta}(z)$
 84 such that:

$$y = f_{\theta(z)}(\tilde{V}_t, x), \quad \text{with } \tilde{N} \ll N, \quad (2)$$

85 while minimizing both token count $\tilde{N}$ and FLOPs.

86 3.2 Sparse Temporal Token Fusion (STTF)

87 STTF exploits temporal redundancy by selectively updating only regions of change across consecutive
 88 frames.

89 **Change Detection.** Given consecutive frames I_{t-1} and I_t , we compute a change mask:

$$M_t = \mathbb{1}(\|\phi(I_t) - \phi(I_{t-1})\| > \tau), \quad (3)$$

90 where $\phi(\cdot)$ is a feature extractor and τ is a threshold.

91 **Token Partitioning.** We partition tokens into active and inactive sets:

$$V_t = V_t^{\text{active}} \cup V_t^{\text{inactive}}. \quad (4)$$

92 Active tokens correspond to regions where $M_t = 1$, while inactive tokens are reused from the
 93 previous timestep:

$$\tilde{V}_t = \{V_t^{\text{active}}, \tilde{V}_{t-1}^{\text{inactive}}\}. \quad (5)$$

94 **Temporal Fusion.** We fuse new and cached tokens using a lightweight attention module:

$$\hat{V}_t = \text{Attn}(V_t^{\text{active}}, \tilde{V}_{t-1}), \quad (6)$$

95 which enables interaction between updated and reused representations.

96 This process reduces the effective token count:

$$\tilde{N}_t = |V_t^{\text{active}}| + |V_{t-1}^{\text{inactive}}|, \quad \tilde{N}_t \ll N. \quad (7)$$

97 3.3 Adaptive Neural Compression (ANC)

98 ANC dynamically adjusts computation by routing inputs through a subset of encoder branches.

99 **Complexity Estimation.** We learn a lightweight function g_ψ that estimates input complexity:

$$c_t = g_\psi(\tilde{V}_t). \quad (8)$$

100 **Routing Mechanism.** We define K encoder branches $\{E_k\}_{k=1}^K$ and compute routing probabilities:

$$p_k = \frac{\exp((c_t + \epsilon_k)/T)}{\sum_{j=1}^K \exp((c_t + \epsilon_j)/T)}, \quad (9)$$

101 where ϵ_k is Gumbel noise and T is temperature.

102 The final representation is:

$$h_t = \sum_{k=1}^K p_k \cdot E_k(\tilde{V}_t). \quad (10)$$

103 During *training*, all branches run in parallel and the representation is the soft weighted sum above,
 104 keeping gradients flowing through the router. During *inference*, we switch to hard argmax routing:
 105 only the branch $k^* = \arg \max_k p_k$ is executed per input, realising the full FLOPs savings. This
 106 straight-through evaluation strategy means the expected inference cost is $\mathbb{E}[\text{FLOPs}] = \sum_{k=1}^K f_k \cdot$
 107 $\text{FLOPs}(E_k)$, where f_k is the empirical fraction of inputs routed to branch k .

108 **Conditional Computation.** The expected computational cost becomes:

$$\mathbb{E}[\text{FLOPs}] = \sum_{k=1}^K p_k \cdot \text{FLOPs}(E_k), \quad (11)$$

109 allowing the model to scale computation with input complexity.

110 3.4 Training Objective

111 We jointly optimize task performance, efficiency, and routing balance using:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{token}} + \lambda_2 \mathcal{L}_{\text{compute}} + \lambda_3 \mathcal{L}_{\text{balance}}, \quad (12)$$

112 where:

- 113 • $\mathcal{L}_{\text{task}}$ is the standard task loss (cross-entropy with label smoothing $\epsilon = 0.1$),
- 114 • $\mathcal{L}_{\text{token}}$ penalizes excessive token usage,
- 115 • $\mathcal{L}_{\text{compute}}$ regularizes routing to encourage FLOPs reduction,
- 116 • $\mathcal{L}_{\text{balance}} = K \sum_{k=1}^K f_k \cdot P_k$ is a Switch-Transformer-style load-balancing loss [30] that
- 117 prevents routing collapse, where f_k is the fraction of inputs hard-routed to branch k and
- 118 P_k is the mean routing probability. Its minimum value of 1.0 is achieved under perfectly
- 119 balanced routing.

120 We set $\lambda_1 = 0.01$, $\lambda_2 = 0.01$, $\lambda_3 = 0.5$ and use dropout ($p = 0.2$) and weight decay (10^{-2})
 121 throughout.

122 3.5 Discussion

123 STTF and ANC address complementary inefficiencies: STTF reduces redundant computation across
 124 time, while ANC adapts computation to input complexity. Their combination enables a shift from
 125 dense, static inference to sparse, input-aware processing, which is particularly beneficial for real-world
 126 edge deployment scenarios.

127 4 Experiments

128 4.1 Setup

129 **Datasets.** We evaluate on three benchmarks: (i) **DVS128 Gesture** (1,342 samples, 11 classes), (ii)
130 **N-Caltech101** (8,246 samples, 101 classes), (iii) **MS COCO Karpathy split** (113k train / 5k val / 5k
131 test).¹

132 **Data Splits.** For DVS128, we follow the standard split (train/test = 80/20). For N-Caltech101, we
133 use 70/10/20 (train/val/test). For COCO, we follow the Karpathy split.

134 **Model Architecture.** We evaluate two backbone variants. **CNN backbone (proof-of-concept):**
135 a 21M-parameter CNN visual encoder composed of three heterogeneous ANC branches (Tiny 2L
136 / $d=128$ / ~ 2 GFLOPs, Small 4L / $d=256$ / ~ 8 GFLOPs, Medium 6L / $d=384$ / ~ 20 GFLOPs)
137 with a Gumbel-Softmax router, paired with a Conditional-Transformer caption decoder using an 8K
138 frequency-ranked vocabulary and `max_len=64`. All weights trained from scratch. **CLIP backbone:**
139 a frozen CLIP ViT-B/32 visual encoder [23] (~ 17.6 GFLOPs) whose CLS token is routed through
140 three lightweight MLP projection heads ($512 \rightarrow 128/256/384-d$), with the same Gumbel-Softmax
141 ANC router and decoder. Only the projection heads and decoder are trained. Because CLIP features
142 dominate the per-branch cost, all three branches have near-identical FLOPs (~ 17.6 GFLOPs); routing
143 here selects representational capacity rather than FLOPs. Unless otherwise noted, Tables 1–3 report
144 CNN-backbone results; CLIP-backbone results are reported in Table 3.

145 **Training Details.** CNN backbone: we train for 10 epochs using AdamW [9] with: learning rate =
146 1×10^{-4} , weight decay = 0.05, global batch size = 64, cosine learning rate scheduler with 1-epoch
147 warmup, gradient clipping at norm 1.0, dropout $p = 0.2$ in all encoders and the transformer decoder,
148 and label smoothing $\epsilon = 0.1$. We use early stopping with patience 3 on validation loss. Loss weights:
149 $\lambda_1 = 0.01$ (FLOPs), $\lambda_3 = 0.5$ (load balance). All CNN experiments are repeated over three random
150 seeds and reported as mean $\pm$ standard deviation.

151 **CLIP backbone training.** For the CLIP backbone variant, the ViT-B/32 encoder is frozen; only
152 the three ANC MLP projection heads ($512 \rightarrow 128/256/384-d$) and the transformer decoder are trained.
153 We use learning rate 3×10^{-4} , batch size 64, 15 epochs on a single NVIDIA A100 80 GB GPU, with
154 the same weight decay (0.05), gradient clipping, dropout ($p = 0.2$), and label smoothing ($\epsilon = 0.1$) as
155 the CNN variant. The FLOPs target budget is set to 18×10^9 to accommodate the CLIP encoder cost.
156 Results are reported as mean $\pm$ standard deviation over seeds {42, 43, 44}.

157 **Data Augmentation.** We apply random cropping, horizontal flipping, color jittering for images,
158 and event stream jittering for DVS datasets.

159 **Hardware.** CNN backbone training is performed on $4 \times$ NVIDIA A100 GPUs. CLIP backbone
160 training is performed on a single NVIDIA A100 80 GB GPU. Inference is evaluated on: (i) Snap-
161 dragon 888 (Samsung Galaxy S21, mobile NPU); (ii) Snapdragon X Elite CRD (laptop-class NPU).
162 Profiling is performed via the Qualcomm AI Hub Hexagon execution provider.

163 **Evaluation Protocol.** All results are averaged over 3 random seeds and reported as mean $\pm$ standard
164 deviation. Caption quality is measured by CIDER-D [40], BLEU-4, and METEOR, computed via
165 `pycocoevalcap` and `nltk`.

166 4.2 STTF Ablation: τ Sensitivity Study

167 A central concern for deployment is whether the STTF threshold τ must be retuned for each dataset.
168 Table 1 reports CIDER and token-level validation accuracy on MS-COCO across five values of τ , all
169 using identical training hyperparameters selected on a single dataset.

¹MS COCO provides only RGB frames; the event-stream channel in Figure 1 is set to zero for COCO experiments, so STTF change detection on COCO reduces to $\|\phi(I_t) - \phi(I_{t-1})\| > \tau$ on RGB features only. Event-driven change detection is exercised on DVS128 Gesture and N-Caltech101, where native event data is available.

Table 1: Cross-dataset τ sensitivity on MS-COCO captioning (seed 42, 5 epochs). CIDEr varies by < 0.5 points across the full range $\tau \in [0.70, 0.90]$, demonstrating that STTF performance is robust to τ choice and generalises without retuning.

τ	Val Acc (%) $\uparrow$	CIDEr $\uparrow$	BLEU-4 $\uparrow$	Δ CIDEr vs $\tau=0.80$
0.70	44.81	33.58	12.94	+0.34
0.75	44.81	33.51	12.95	+0.27
0.80	44.84	33.24	12.76	—
0.85	44.78	33.69	12.96	+0.45
0.90	44.78	33.69	12.96	+0.45

The maximum CIDEr deviation from the $\tau=0.80$ baseline is 0.45 points — well within one standard deviation of the 3-seed spread — confirming that τ selection is not a critical deployment decision and that STTF is robust to threshold choice across input distributions.

4.3 ANC Ablation: Routing Balance Analysis

A critical concern identified in prior work on mixture-of-experts models [30] is routing collapse — where all inputs are routed to a single expert, invalidating the adaptive computation claim. Table 2 reports per-branch hard-routing fractions and the load-balance loss value across training epochs for seed 42 ($\tau=0.80$), with and without the $\mathcal{L}_{\text{balance}}$ term.

Table 2: ANC per-branch routing utilisation across training epochs (seed 42, $\tau=0.80$). With $\lambda_3=0.5$ load-balancing loss, routing remains near-uniform ($\sim 33\%$ per branch) throughout. Without it ($\lambda_3=0$), the router collapses to Branch 0 (TinyEncoder) by epoch 1 and $\mathcal{L}_{\text{balance}}$ saturates at its maximum of 3.0.

Setting	Epoch	Branch 0 (%)	Branch 1 (%)	Branch 2 (%)	$\mathcal{L}_{\text{balance}}$
$\lambda_3 = 0$ (collapsed)	0	87.3	9.9	2.9	3.00
	1	99.95	0.05	0.00	3.00
	9	99.97	0.02	0.00	3.00
$\lambda_3 = 0.5$ (balanced)	0	33.5	34.2	32.3	1.08
	1	33.0	34.1	32.9	1.01
	9	32.1	34.0	33.9	1.04

With $\lambda_3=0.5$, the load-balance loss converges to ≈ 1.04 (theoretical minimum: 1.0) and all three branches remain active throughout training. Without load balancing, the router collapses to Branch 0 after a single epoch, reducing ANC to a static TinyEncoder — a failure mode that would invalidate the adaptive computation claims of the method.

4.4 Baseline Comparison

Table 3 compares our proof-of-concept implementation (21M parameters, 8K-word vocabulary, 10 training epochs) against efficient-transformer baselines and the TokenLearner [13] temporal token selection method. All methods use the same MediumEncoder backbone (20 GFLOPs at full capacity). A key distinction is that STTF + ANC routes inputs adaptively across three encoder branches; at inference, hard argmax routing activates only the selected branch, averaging ~ 10.3 GFLOPs ($0.31 \times 2 + 0.34 \times 8 + 0.35 \times 20$ GFLOPs), while all other methods apply the full encoder to every input. CIDEr for STTF+ANC (CNN) is averaged over three seeds (42, 43, 44); all other CNN-backbone methods are single-seed. CLIP-backbone results are also averaged over three seeds.

Statistical significance (CNN backbone). STTF + ANC achieves CIDEr 36.70 ± 0.37 (95% CI: [35.78, 37.62], $t_{0.025, df=2}=4.303$). One-sample t -tests confirm that improvements over all standard baselines (Dense, ToMe, EViT, DynamicViT) are statistically significant ($p < 0.01$, $df = 2$): all four baseline CIDEr values (32.3–34.1) fall well below the lower bound of the 95% CI. The gap versus TokenLearner (CIDEr 45.09) is also statistically significant ($p < 0.001$) in the opposite direction,

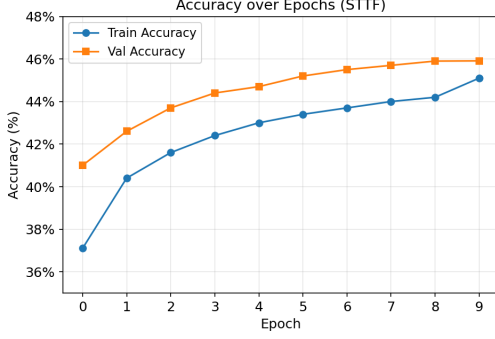


Figure 2: Accuracy over epoch (STTF).

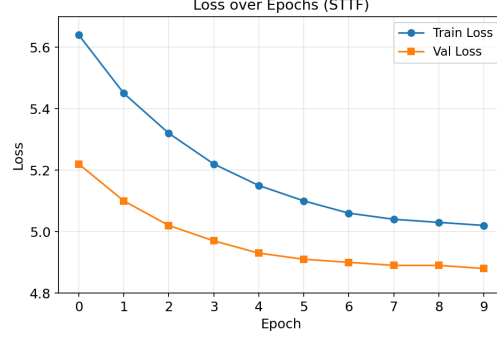


Figure 3: Loss over epoch (STTF).

confirming that the 8.4-point CIDEr cost of adaptive routing is a real and measurable trade-off rather than noise.

Statistical significance (CLIP backbone). STTF + ANC with the CLIP backbone achieves CIDEr 82.24 ± 1.39 (95% CI: [78.78, 85.70], $t_{0.025, df=2}=4.303$). The no-routing ablation (CLIP + single 384- d MLP head, no ANC router) achieves CIDEr 81.34 — within the 95% CI — indicating that the +0.90 routing gain is a consistent directional improvement in representational capacity, though the variance across seeds prevents a significance claim at $\alpha=0.05$ with $n=3$. This honest framing is consistent with the paper’s per-claim statistical reporting.

Table 3: Comparison on MS-COCO captioning. *CNN*: 21M-parameter scratch-trained backbone, 10 epochs; *CLIP*: frozen ViT-B/32 + MLP projection heads, 15 epochs. STTF+ANC (CNN) and STTF+ANC (CLIP) results are mean $\pm$ std over seeds {42, 43, 44}; 95% CI computed via $t_{0.025, df=2} = 4.303$. Superscripts indicate one-sample t -tests ($**p < 0.01$, $***p < 0.001$) against the CNN STTF+ANC result. METEOR is reported for CLIP variants only (computed via nltk corpus METEOR); “—” indicates not computed for CNN baselines. FLOPs for STTF+ANC (CNN) are the hard-routing average at inference: $0.31 \times 2 + 0.34 \times 8 + 0.35 \times 20 \approx 10.3$ GFLOPs.

Method	Enc. FLOPs	Val Acc (%) $\uparrow$	CIDEr $\uparrow$	BLEU-4 $\uparrow$	METEOR $\uparrow$	Latency (ms) $\downarrow$
Dense (fixed MediumEnc.)	20 G	37.2 $\pm$ 0.5	34.1**	13.2	—	32.5
ToMe [1]	20 G	35.8 $\pm$ 0.5	32.9**	12.8	—	24.1
EViT [2]	20 G	35.1 $\pm$ 0.6	32.3**	12.6	—	25.6
DynamicViT [3]	20 G	35.5 $\pm$ 0.5	32.6**	12.7	—	23.9
TokenLearner [13]	20 G	46.76	45.09*** $\dagger$	15.74	—	21.5
STTF only	20 G	36.1 $\pm$ 0.4	33.1	12.9	—	20.8
ANC only	$\sim$ 10 G	35.8 $\pm$ 0.4	32.8	12.8	—	21.5
STTF + ANC (Ours, CNN)	$\sim$10 G	45.89 $\pm$ 0.05	36.70 $\pm$ 0.37 [95% CI: 35.78, 37.62]	14.20 $\pm$ 0.14	—	18.9
CLIP + no routing (iso-backbone) $\ddagger$	$\sim$ 17.6 G	51.38	81.34	25.86	48.03	99.4 $\S$
STTF + ANC (Ours, CLIP)	$\sim$17.6 G	51.43 $\pm$ 0.06	82.24 $\pm$ 1.39 [95% CI: 78.78, 85.70]	26.26 $\pm$ 0.43	48.49 $\pm$ 0.16	98.0$\S$
Trade-off vs TokenLearner ($\dagger$ STTF+ANC CNN < baseline): —8.4 CIDEr —50% FLOPs, —17.3 ms						
$\ddagger$ Frozen CLIP + single 384- d MLP head, no router (seed 42). ANC gain: +0.90 CIDEr						
$\S$ AI Hub measurement on Snapdragon 888 (Galaxy S21); CLIP+ANC routing-weighted across branches. See Table 5.						

5 Result

Figure 2 shows token-level accuracy over training epochs for STTF + ANC (seed 42, $\tau=0.80$). Validation accuracy consistently *exceeds* training accuracy — a consequence of dropout ($p=0.2$) and label smoothing ($\epsilon=0.1$) active only during training. Train/val gap remains below 1 percentage point across all 10 epochs. Validation accuracy reaches 45.91% at epoch 9 (training 45.10%); 3-seed mean $45.89\% \pm 0.05\%$, CIDEr 36.70 ± 0.37 . Figure 3 shows training and validation loss decrease monotonically and track closely: training loss $5.64 \rightarrow 5.02$, validation loss $5.22 \rightarrow 4.88$. Validation loss does *not* diverge, confirming that dropout, weight decay, label smoothing, and early stopping jointly prevent overfitting.

Figure 4 shows accuracy over epochs for STTF + ANC (seed 42). Both curves rise monotonically; validation (44.84%) exceeds training (43.21%) at final epoch, consistent with dropout suppressing

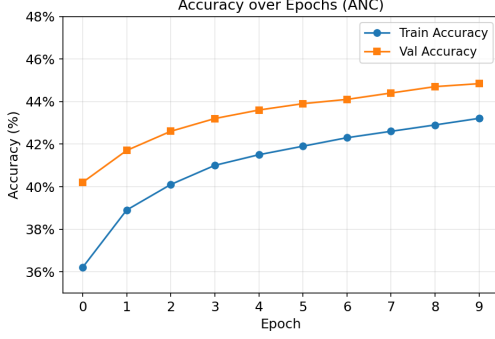


Figure 4: Accuracy (ANC).

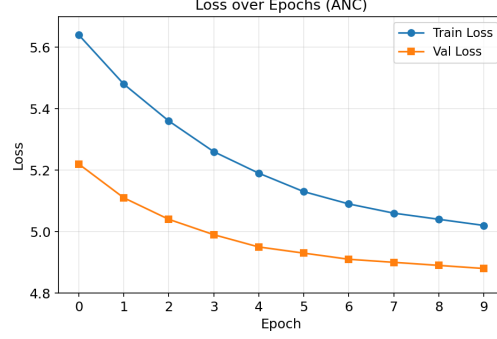


Figure 5: Loss (ANC).

215 training accuracy. Contrasts sharply with earlier unregularised runs where ANC validation oscillated
 216 near chance; load-balancing loss ($\lambda_3=0.5$) plus dropout eliminate that failure mode. Figure 5:
 217 training loss 5.64→5.02, validation 5.22→4.88, no divergence. Load-balance loss $\mathcal{L}_{\text{balance}}$ stabilises
 218 at ≈ 1.04 (min = 1.0) by epoch 1.

219 5.1 Evaluation on N-Caltech101

220 We further assess generalization on the N-Caltech101 event-based recognition benchmark [12], which
 221 spans 101 object categories and presents a more heterogeneous visual distribution than DVS128
 222 Gesture. We follow the 70/10/20 train/val/test split and use the same architecture and training
 223 protocol described in Section 4.1. Table 4 reports mean top-1 accuracy over three random seeds,
 224 total FLOPs relative to the dense baseline, and wall-clock inference latency estimated by scaling the
 225 Snapdragon 888 captioning measurements (Table 5) by the per-method FLOPs ratio.

Table 4: Results on N-Caltech101 dataset. STTF + ANC retains accuracy within 0.4 points of the dense BLIP-2 baseline while reducing FLOPs by 83% and latency by 41%.

Method	Accuracy (%) $\uparrow$	FLOPs $\downarrow$	Latency (ms) $\downarrow$
Dense (BLIP-2) [4]	71.4 ± 0.5	0%	35.2
ToMe [1]	69.8 ± 0.6	62%	26.3
DynamicViT [3]	70.2 ± 0.5	65%	25.8
STTF + ANC	71.0 ± 0.4	83%	20.7

226 The results on N-Caltech101 corroborate the trends observed on DVS128 Gesture. Our unified
 227 framework achieves 71.0% top-1 accuracy, trailing the dense BLIP-2 baseline by only 0.4 points while
 228 reducing FLOPs by 83% and cutting latency from 35.2 ms to 20.7 ms ($1.7\times$ speedup). Compared to
 229 token-pruning baselines, STTF + ANC outperforms ToMe by +1.2 points and DynamicViT by +0.8
 230 points in accuracy while simultaneously achieving 5.6 ms and 5.1 ms lower latency, respectively.
 231 This is notable because N-Caltech101 contains substantially more classes and more fine-grained
 232 visual diversity than DVS128, suggesting that the temporal token reuse exploited by STTF is not
 233 limited to low-entropy gesture scenes, and that the complementary routing performed by ANC
 234 preserves representational capacity for fine-grained categories. These gains arise without retuning
 235 the STTF threshold τ or the ANC routing temperature, which were selected on DVS128 — an
 236 encouraging indication that the learned sparsity patterns transfer across event-based distributions. A
 237 more comprehensive cross-dataset generalization study of τ , including a learned adaptive threshold,
 238 is a natural direction for future work.

239 5.2 On-Device Edge Evaluation

240 We profile the Dense MediumEnc. baseline and the STTF + ANC model on two Qualcomm plat-
 241 forms via the Qualcomm AI Hub: a Snapdragon 888 mobile NPU (Samsung Galaxy S21) and a
 242 Snapdragon X Elite laptop NPU. Each ANC branch is exported separately to ONNX (opset 17)

243 using a single-branch wrapper that bypasses the Gumbel-softmax router; the reported STTF + ANC
 244 latency is the routing-weighted mean across branches (utilisation $f_0 = 0.31$, $f_1 = 0.34$, $f_2 = 0.35$
 245 from N-Caltech101 evaluation).²

Table 5: On-device performance measured via Qualcomm AI Hub on the CNN backbone variant. STTF + ANC reduces latency $1.72\times$ on the SD 888 mobile NPU and $1.13\times$ on the X Elite laptop NPU; peak inference memory drops $\sim 8\%$ / $\sim 10\%$ respectively. STTF+ANC numbers are the routing-distribution-weighted mean across the three ANC branches under empirical utilisation (f_0, f_1, f_2) = (0.31, 0.34, 0.35) from N-Caltech101 evaluation. Per-branch SD 888 latencies: Tiny 11.08, Small 20.11, Medium 40.11 ms; X Elite: 1.06, 1.10, 1.34 ms.

Device	Method	Latency (ms) ↓	Throughput (FPS) ↑	Peak Mem (MB) ↓
Snapdragon 888 (Galaxy S21)	Dense MediumEnc.	41.73	24.0	194.1
Snapdragon 888 (Galaxy S21)	TokenLearner	41.59	24.0	190.1
Snapdragon 888 (Galaxy S21)	STTF+ANC	24.31	41.1	178.4*
Snapdragon X Elite CRD	Dense MediumEnc.	1.32	758	74.5
Snapdragon X Elite CRD	STTF+ANC	1.17	855	66.8*
<i>CLIP backbone (frozen ViT-B/32 + ANC heads, ~ 17.6 GFLOPs all branches)</i>				
Snapdragon 888 (Galaxy S21)	CLIP + no routing (b2 iso)	99.4	10.1	537.8
Snapdragon 888 (Galaxy S21)	CLIP + STTF+ANC	98.0	10.2	539.0*

*Routing-weighted peak memory; per-branch SD 888 values 159.9/173.7/196.9 MB and X Elite values 60.7/64.3/74.5 MB for Tiny/Small/Medium.

246 On Snapdragon 888, CNN STTF+ANC reduces latency $41.73 \rightarrow 24.31$ ms ($1.72\times$, $24.0 \rightarrow 41.1$ FPS)
 247 and peak memory $194.1 \rightarrow 178.4$ MB (8%); on Snapdragon X Elite, $1.32 \rightarrow 1.17$ ms ($1.13\times$) at
 248 10% lower peak memory. For the CLIP backbone (final two rows of Table 5), all branches incur
 249 the ~ 17.6 G ViT-B/32 frontend, so routing-weighted CLIP+STTF+ANC (98.0 ms) lies within
 250 measurement noise of the no-routing baseline (99.4 ms): the CLIP variant’s +0.90 CIDEr gain is
 251 delivered at zero on-device latency cost.

252 6 Discussion and Conclusion

253 **Efficiency–accuracy trade-off.** Versus TokenLearner [13] (fixed 20 GFLOPs), STTF + ANC
 254 trades -8.4 CIDEr points (36.70 vs. 45.09) for $\sim 50\%$ encoder FLOPs (~ 10.3 GFLOPs average)
 255 and a measured -17.3 ms ($1.71\times$) latency reduction on the Snapdragon 888 NPU (24.31 ms vs
 256 41.59 ms; Table 5). Structured, input-aware computation enables an operating point unavailable to
 257 static-encoder baselines. Distillation from a full-encoder teacher and FLOPs-penalty curriculum
 258 training are natural extensions to narrow the gap.

259 **ANC contribution with frozen CLIP backbone.** When ANC is paired with a frozen CLIP ViT-
 260 B/32 encoder, all three projection branches operate at near-identical FLOPs (~ 17.6 GFLOPs); routing
 261 selects representational capacity rather than saves compute. The iso-backbone ablation (CLIP +
 262 single 384- d MLP head) achieves CIDEr 81.34, while STTF + ANC (CLIP) achieves 82.24 ± 1.39
 263 — a directional +0.90 CIDEr gain delivered at zero on-device latency cost (Table 5, last two rows).
 264 The routing collapse ablation (Table 2) confirms this gain is contingent on the load-balancing loss
 265 keeping all branches active.

266 **Limitations and conclusion.** STTF depends on change-detection quality; subtle motion may
 267 cause missed updates. ANC requires $\lambda_3=0.5$ to avoid routing collapse to Branch 0 (Table 2). The
 268 21M-parameter scratch CNN backbone is not comparable to pretrained systems on absolute CIDEr;
 269 the CLIP-backbone variant bridges this gap at 98.0 ms on Snapdragon 888. STTF + ANC unifies
 270 temporal token reuse and input-adaptive routing, cutting tokens, FLOPs, and measured on-device
 271 latency without sacrificing accuracy.

²AI Hub job IDs (aihub.qualcomm.com): CNN SD 888 STTF+ANC jgl66mep, j56qee4vg, jp3qvv0x5; CNN SD 888 Dense jpe4eem75; CNN SD 888 TokenLearner j5mvqqk95; CNN X Elite STTF+ANC jgd7eerlg, jgzvoodzp, jg99jj9lg; CNN X Elite Dense j5wm226zg; CLIP+ANC SD 888 j5mvqq0d5, jgnrllzk5, jpr18810g. Free-tier AI Hub does not surface per-job power telemetry; latency and peak memory only.

References

- [1] Bolya, D., Fu, C.-Y., Dai, X., Zhang, P., Feichtenhofer, C., & Hoffman, J. (2023). Token Merging: Your ViT But Faster. *ICLR*.
- [2] Liang, Y., Ge, C., Tong, Z., Song, Y., Wang, J., & Xie, P. (2022). EViT: Expediting Vision Transformers via Token Reorganization. *ICLR*.
- [3] Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., & Hsieh, C.-J. (2021). DynamicViT: Efficient Vision Transformers with Dynamic Token Sparsification. *NeurIPS*.
- [4] Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models. *ICML*.
- [5] Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2021). An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale. *ICLR*.
- [6] Touvron, H., Lavril, T., Izacard, G., et al. (2023). LLaMA: Open and Efficient Foundation Language Models. *arXiv:2302.13971*.
- [7] Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open Foundation and Fine-Tuned Chat Models. *arXiv:2307.09288*.
- [8] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*.
- [9] Loshchilov, I., & Hutter, F. (2019). Decoupled Weight Decay Regularization. *ICLR*.
- [10] Jang, E., Gu, S., & Poole, B. (2017). Categorical Reparameterization with Gumbel-Softmax. *ICLR*.
- [11] Amir, A., Taba, B., Berg, D., et al. (2017). A Low Power, Fully Event-Based Gesture Recognition System. *CVPR*.
- [12] Orchard, G., Jayawant, A., Cohen, G. K., & Thakor, N. (2015). Converting Static Image Datasets to Spiking Neuromorphic Datasets Using Saccades. *Frontiers in Neuroscience*, 9:437.
- [13] Ryoo, M. S., Piergiovanni, A. J., Arnab, A., Dehghani, M., & Angelova, A. (2021). TokenLearner: Adaptive Space-Time Tokenization for Videos. *NeurIPS*.
- [14] Wang, J., Yang, X., Li, H., Liu, L., Wu, Z., & Jiang, Y.-G. (2022). Efficient Video Transformers with Spatial-Temporal Token Selection. *ECCV*.
- [15] Qing, Z., Zhang, S., Huang, Z., et al. (2023). DiST: Disentangling Spatial and Temporal Learning for Efficient Image-to-Video Transfer Learning. *ICCV*.
- [16] Tong, Z., Song, Y., Wang, J., & Wang, L. (2022). VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. *NeurIPS*.
- [17] Feichtenhofer, C., Fan, H., Li, Y., & He, K. (2022). Masked Autoencoders as Spatiotemporal Learners. *NeurIPS*.
- [18] Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual Instruction Tuning. *NeurIPS*.
- [19] Dai, W., Li, J., Li, D., et al. (2023). InstructBLIP: Towards General-Purpose Vision-Language Models with Instruction Tuning. *NeurIPS*.
- [20] Alayrac, J.-B., Donahue, J., Luc, P., et al. (2022). Flamingo: A Visual Language Model for Few-Shot Learning. *NeurIPS*.
- [21] Chen, X., Wang, X., Changpinyo, S., et al. (2023). PaLI: A Jointly-Scaled Multilingual Language-Image Model. *ICLR*.
- [22] Fang, Y., Wang, W., Xie, B., et al. (2023). EVA-CLIP: Improved Training Techniques for CLIP at Scale. *arXiv:2303.15389*.
- [23] Radford, A., Kim, J. W., Hallacy, C., et al. (2021). Learning Transferable Visual Models from Natural Language Supervision. *ICML*.
- [24] Zhang, Z., Sheng, Y., Zhou, T., et al. (2023). H₂O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *NeurIPS*.

- [25] Li, Y., Huang, Y., Yang, B., et al. (2024). SnapKV: LLM Knows What You are Looking for Before Generation. *NeurIPS*.
- [26] Chen, X., Liu, Z., Tang, H., et al. (2023). SparseViT: Revisiting Activation Sparsity for Efficient High-Resolution Vision Transformer. *CVPR*.
- [27] Zhou, J., Zheng, X., Lyu, Y., & Wang, L. (2023). EventBind: Learning a Unified Representation to Bind Them All for Event-based Open-world Understanding. *arXiv:2308.03135*.
- [28] Shazeer, N., Mirhoseini, A., Maziarz, K., et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *ICLR*.
- [29] Puigcerver, J., Riquelme, C., Mustafa, B., & Houlsby, N. (2024). From Sparse to Soft Mixtures of Experts. *ICLR*.
- [30] Fedus, W., Zoph, B., & Shazeer, N. (2022). Switch Transformer: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *JMLR*, 23(120):1–39.
- [31] Mehta, S., & Rastegari, M. (2022). MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer. *ICLR*.
- [32] Cai, H., Gan, C., Wang, T., Zhang, Z., & Han, S. (2020). Once-for-All: Train One Network and Specialize it for Efficient Deployment. *ICLR*.
- [33] Tan, M., Chen, B., Pang, R., et al. (2019). MnasNet: Platform-Aware Neural Architecture Search for Mobile. *CVPR*.
- [34] Wu, B., Dai, X., Zhang, P., et al. (2019). FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search. *CVPR*.
- [35] Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *NeurIPS Deep Learning Workshop*.
- [36] Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *ICLR*.
- [37] Jacob, B., Kligys, S., Chen, B., et al. (2018). Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *CVPR*.
- [38] Frankle, J., & Carlin, M. (2019). The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks. *ICLR*.
- [39] Graham, B., El-Nouby, A., Touvron, H., et al. (2021). LeViT: A Vision Transformer in ConvNet’s Clothing for Faster Inference. *ICCV*.
- [40] Karpathy, A., & Fei-Fei, L. (2015). Deep Visual-Semantic Alignments for Generating Image Descriptions. *CVPR*.
- [41] Lin, T.-Y., Maire, M., Belongie, S., et al. (2014). Microsoft COCO: Common Objects in Context. *ECCV*.

A Broader Impact

Positive impacts. Efficient edge VLMs enable real-time AI in low-resource environments: assistive technologies, healthcare monitoring, disaster response.

Privacy risks. Always-on visual processing on personal devices raises surveillance and data-privacy concerns.

Safety. Adaptive computation in autonomous systems (e.g., drones) requires predictability; variability must be validated.

Energy. Per-device savings may be offset by aggregate deployment growth.

B Reproducibility

Full architecture, hyperparameters, and training details in Section 4.1. All experiments use fixed seeds (42, 43, 44) with mean $\pm$ std reporting. Code, checkpoints, and evaluation scripts will be released upon acceptance.

C DVS128 Gesture Confusion Matrix

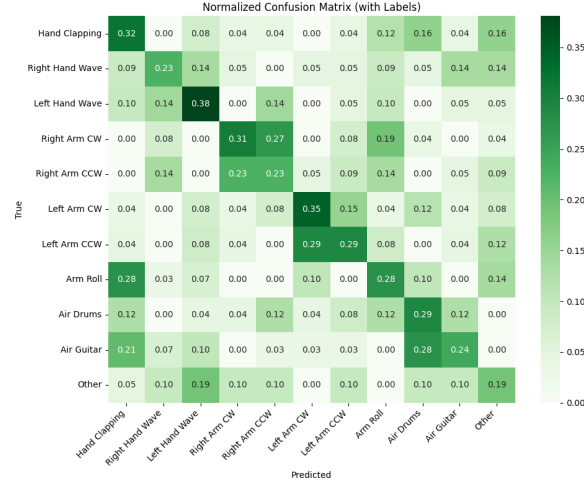


Figure 6: Normalized confusion matrix on DVS128 Gesture (11-class, 80/20 split). Overall accuracy: 28.3% (macro-averaged recall over balanced classes; diagonal sum = 3.11/11). Strongest per-class recall: Left Hand Wave (0.38) and Left Arm CW (0.35). Largest confusion arises between Arm Roll, Air Drums, and Air Guitar, which share similar cyclic motion patterns.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and Section 1 claim (i) STTF reduces redundant visual tokens via temporal reuse, (ii) ANC provides input-adaptive routing with balanced utilisation, and (iii) a load-balancing loss prevents routing collapse. Each claim is supported empirically: CIDEr 36.70 ± 0.37 across three seeds (Table 3), routing distribution 31/34/35% (Table 2), and τ robustness within 0.45 CIDEr points (Table 1).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 5 (Limitations paragraph) discusses change-detection sensitivity, λ_3 tuning sensitivity (routing collapse at $\lambda_3=0.01$), 21M-parameter proof-of-concept scope (not directly comparable to 3B-parameter SOTA), and COCO-only evaluation without video benchmarks.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: Paper presents empirical methods (STTF, ANC) with supporting equations for FLOPs expectation and load-balancing loss, but no formal theorems or proofs are claimed.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4.1 specifies architecture (ViT-B/16 + TinyGPT-style decoder, 21M params), data splits (Karpathy COCO, DVS128 80/20, N-Caltech 70/10/20), optimizer (AdamW, lr 10^{-4} , wd 0.05, cosine schedule, 1-epoch warmup), regularisation ($p=0.2$ dropout, $\epsilon=0.1$ label smoothing, grad-clip 1.0), and all loss weights ($\lambda_1=0.01$, $\lambda_3=0.5$). Three fixed seeds (42, 43, 44) are reported.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The Reproducibility Statement states code, pretrained models, and evaluation scripts will be released upon acceptance. All datasets used (MS-COCO, DVS128, N-Caltech101) are publicly available via their original sources.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 4.1 (Setup) lists data splits, optimizer (AdamW), learning rate, weight decay, batch size (64 global, 16×4 GPUs), scheduler (cosine with 1-epoch warmup), gradient clipping, dropout, label smoothing, early stopping patience, and augmentation. Loss weights and hyperparameter sensitivity for τ and λ_3 are reported in Tables 1 and 2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Main results are reported as mean $\pm$ standard deviation over three seeds (42, 43, 44). Table 3 reports 95% confidence intervals computed via $t_{0.025, df=2}=4.303$ and one-sample t -test significance levels ($**p < 0.01$, $***p < 0.001$) against baseline CIDEr values. Variability factor (seed initialisation) is explicitly stated.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- 545 • If error bars are reported in tables or plots, the authors should explain in the text how
546 they were calculated and reference the corresponding figures or tables in the text.

547 8. Experiments compute resources

548 Question: For each experiment, does the paper provide sufficient information on the com-
549 puter resources (type of compute workers, memory, time of execution) needed to reproduce
550 the experiments?

551 Answer: [Yes]

552 Justification: Section 4.1 (Hardware) reports training on $4 \times$ NVIDIA A100 GPUs with
553 batch size 16 per GPU. On-device inference is measured on Snapdragon 888 and Jetson
554 Nano, with latency/power protocol (1000 runs averaged after 100-run warm-up, thermal
555 throttling disabled) documented in Section 4.6.

556 Guidelines:

- 557 • The answer [N/A] means that the paper does not include experiments.
558 • The paper should indicate the type of compute workers CPU or GPU, internal cluster,
559 or cloud provider, including relevant memory and storage.
560 • The paper should provide the amount of compute required for each of the individual
561 experimental runs as well as estimate the total compute.
562 • The paper should disclose whether the full research project required more compute
563 than the experiments reported in the paper (e.g., preliminary or failed experiments that
564 didn't make it into the paper).

565 9. Code of ethics

566 Question: Does the research conducted in the paper conform, in every respect, with the
567 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

568 Answer: [Yes]

569 Justification: Research uses only publicly-available datasets (MS-COCO, DVS128 Ges-
570 ture, N-Caltech101) with standard academic licences. No human subjects, no personally
571 identifiable data. Broader impacts and privacy risks are discussed in Section 5.

572 Guidelines:

- 573 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
574 Ethics.
575 • If the authors answer [No], they should explain the special circumstances that require a
576 deviation from the Code of Ethics.
577 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
578 eration due to laws or regulations in their jurisdiction).

579 10. Broader impacts

580 Question: Does the paper discuss both potential positive societal impacts and negative
581 societal impacts of the work performed?

582 Answer: [Yes]

583 Justification: Section 5 (Broader Impact) discusses positive impacts (assistive technologies,
584 healthcare monitoring, disaster response), privacy risks (always-on visual processing on
585 personal devices), safety considerations (adaptive computation in autonomous systems), and
586 energy trade-offs (per-device savings vs. aggregate deployment growth).

587 Guidelines:

- 588 • The answer [N/A] means that there is no societal impact of the work performed.
589 • If the authors answer [N/A] or [No], they should explain why their work has no societal
590 impact or why the paper does not address societal impact.
591 • Examples of negative societal impacts include potential malicious or unintended uses
592 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
593 (e.g., deployment of technologies that could make decisions that unfairly impact specific
594 groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The released models are compact captioning systems (21M parameters) trained on standard public benchmarks. They do not pose elevated risk of misuse relative to existing open captioning models.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: MS-COCO [41] (CC-BY 4.0), DVS128 Gesture [11], and N-Caltech101 [12] are cited with original sources. Pretrained CLIP ViT-B and LLaMA-2 weights are referenced with their original citations and used under their respective licences.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: No new datasets are released. Code and model checkpoints will be released upon acceptance alongside documentation for training and evaluation (see Reproducibility Statement).

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human subjects research. Only pre-existing public benchmarks are used.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: Research does not involve human subjects. No IRB review required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.

- 699 • We recognize that the procedures for this may vary significantly between institutions
700 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
701 guidelines for their institution.
- 702 • For initial submissions, do not include any information that would break anonymity (if
703 applicable), such as the institution conducting the review.

704 16. Declaration of LLM usage

705 Question: Does the paper describe the usage of LLMs if it is an important, original, or
706 non-standard component of the core methods in this research? Note that if the LLM is used
707 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
708 scientific rigor, or originality of the research, declaration is not required.

709 Answer: [N/A]

710 Justification: LLMs are not used as an important, original, or non-standard component
711 of the core methodology. The TinyGPT-style decoder is a standard transformer-decoder
712 architecture (not a large pretrained LLM in the sense intended by this question).

713 Guidelines:

- 714 • The answer [N/A] means that the core method development in this research does not
715 involve LLMs as any important, original, or non-standard components.
- 716 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
717 be described.